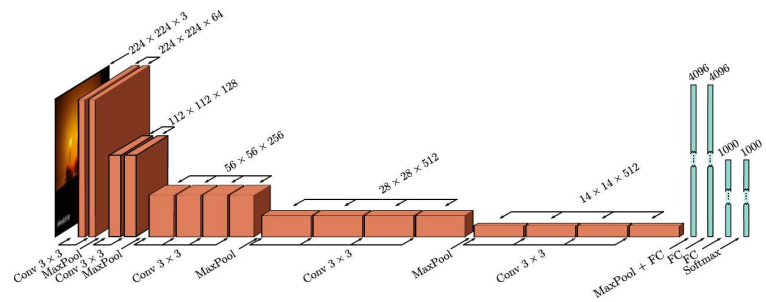
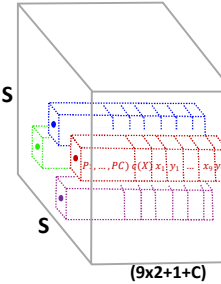
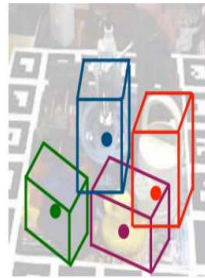


Agenda

- Convolutional Neural Network

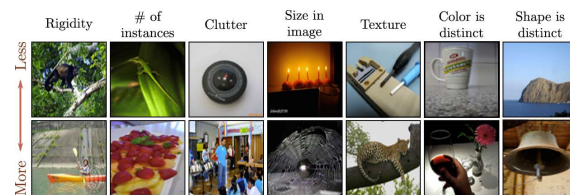
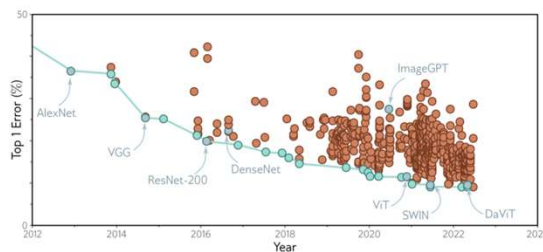


$$N_{\text{out}} = \left\lfloor \frac{N_{\text{in}} + 2P - K}{S} \right\rfloor + 1$$



1

ImageNet



- Alexnet had a 16.4% top-5 error rate
- Within a couple of years of Alexnet, deep(er) neural nets matched human performance (~5.1% Top-5 error rate)
- 224 x 224 images
- 1,281,167 training images, 50,000 validation images, and 100,000 test images
- 1000 classes

2

Earlier multiclass classification

- Predict a score for each class
 - Apply the softmax function
 - Compute the loss function
 - Update the parameters
- At inference output the probability
- But how to predict the score?

$$\hat{y}_j = \frac{e^{-z_j}}{\sum_{k=1}^K e^{-z_k}}$$

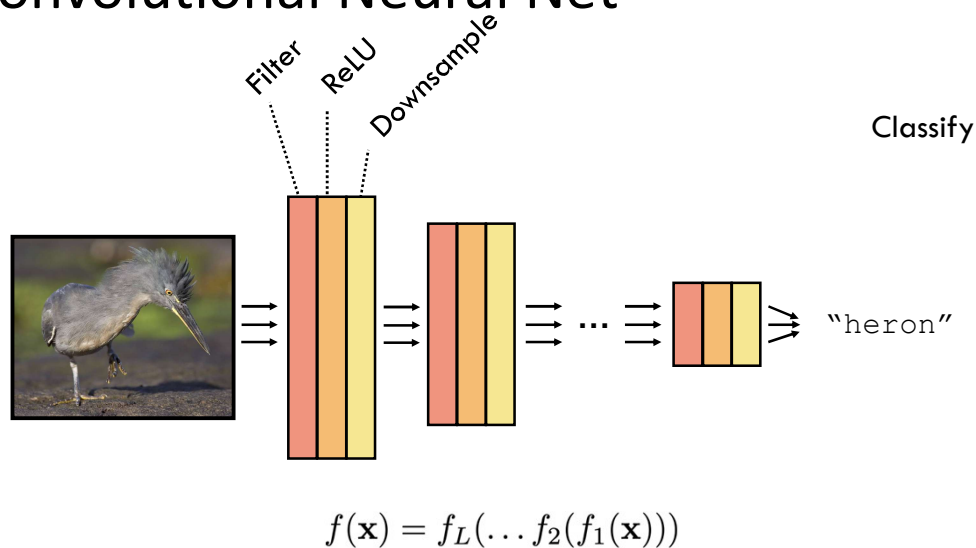
$$\text{softmax}_k[\mathbf{z}] = \frac{\exp[z_k]}{\sum_{k'=1}^K \exp[z_{k'}]}$$

$$\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = H(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{k=1}^K y_k \log \hat{y}_k$$

$$Pr(y = k | \mathbf{x}) = \text{softmax}_k[\mathbf{f}[\mathbf{x}, \phi]]$$

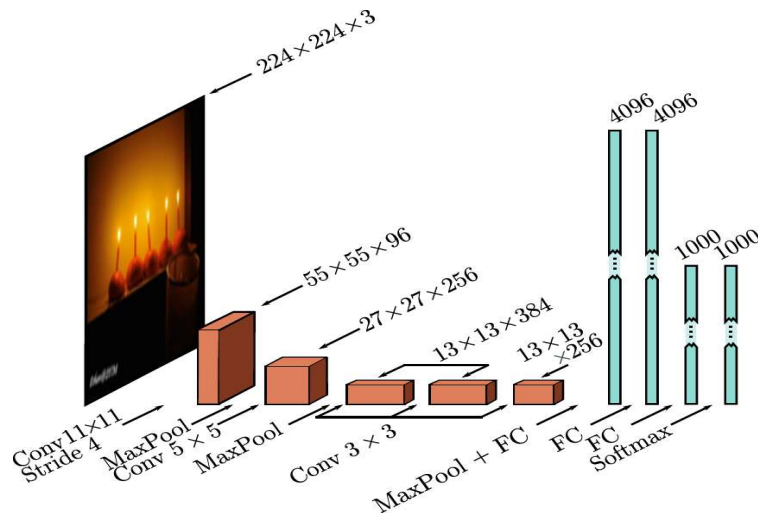
3

Convolutional Neural Net



4

AlexNet (2012)

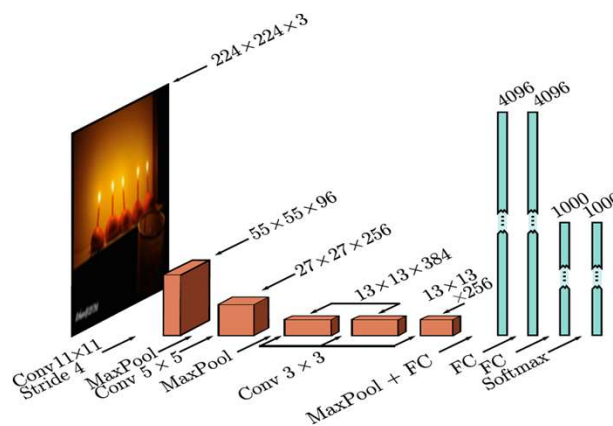


- Almost all the 60 million parameters are in fully connected layers
- 16.4% top-5 error rate and a 38.1% top-1 error rate

$$N_{\text{out}} = \left\lfloor \frac{N_{\text{in}} + 2P - K}{S} \right\rfloor + 1$$

5

Alexnet dimension validation

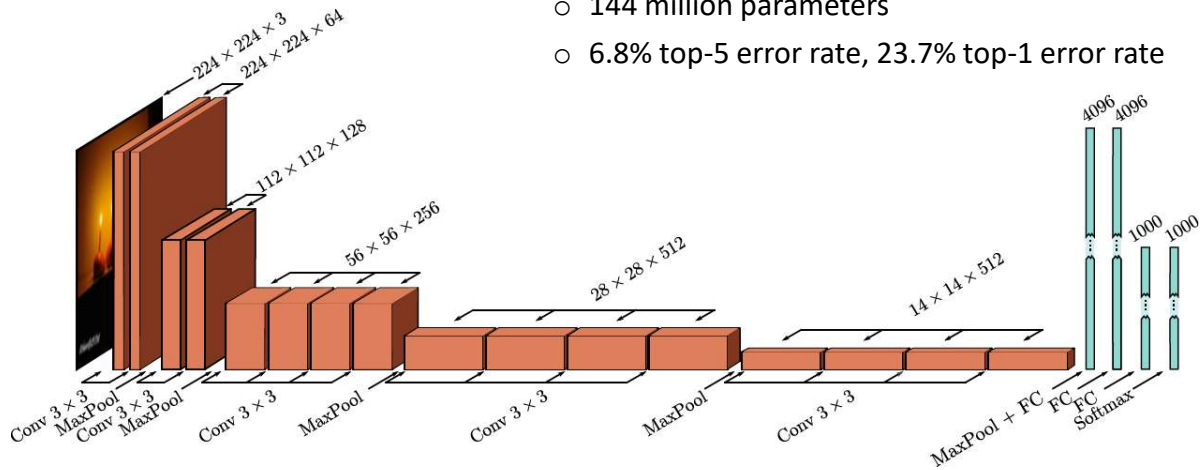


Layer	Input	Kernel	Stride	Padding	Calculation	Output
Conv1	227	11	4	0	$\frac{227-11}{4} + 1 = 55$	55
Pool1	55	3	2	0	$\frac{55-3}{2} + 1 = 27$	27

6

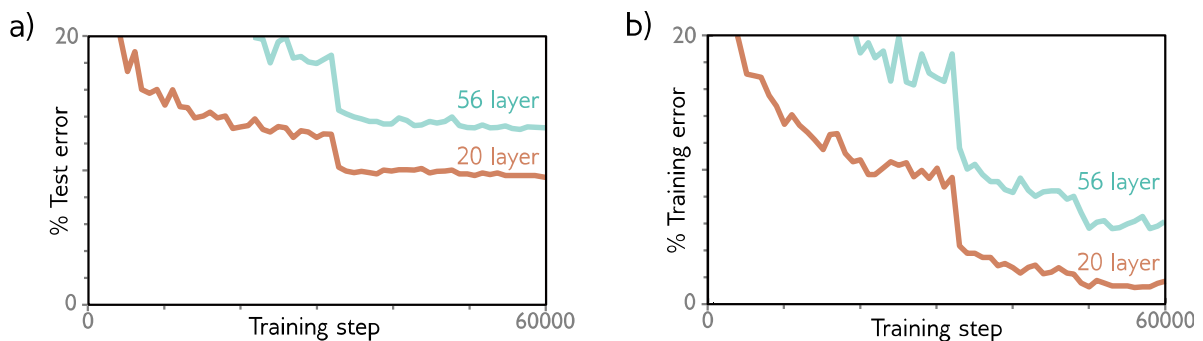
VGG (2015)

- 19 hidden layers
- 144 million parameters
- 6.8% top-5 error rate, 23.7% top-1 error rate



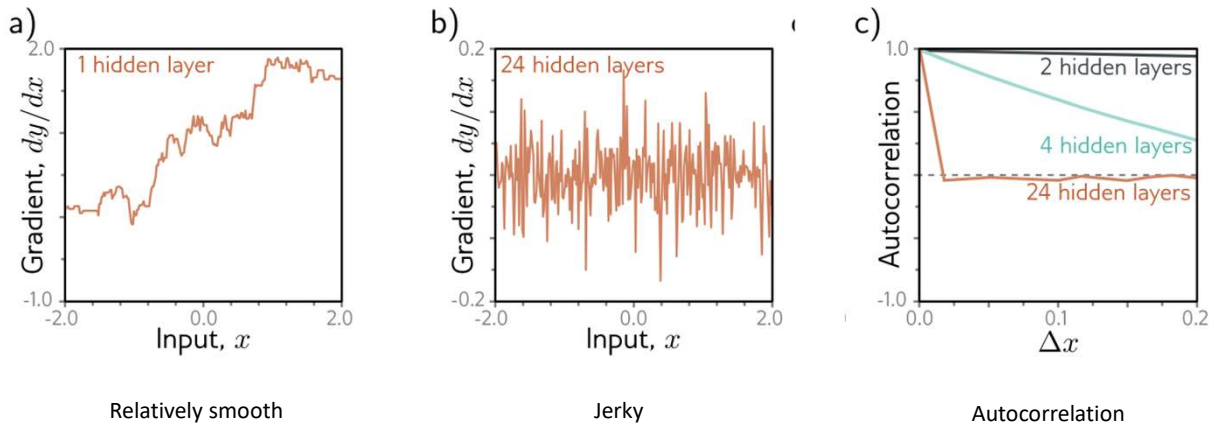
8

Image classification for deeper networks



9

How does the gradient change?



Relatively smooth

Jerky

Autocorrelation

As we increase the number of layers we get jerky behaviour aka "shattered gradient"

10

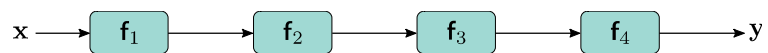
Regular network:

$$h_1 = f_1[x, \phi_1]$$

$$h_2 = f_2[h_1, \phi_2]$$

$$h_3 = f_3[h_2, \phi_3]$$

$$y = f_4[h_3, \phi_4]$$



11

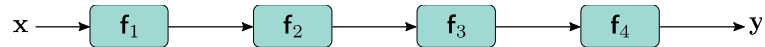
Regular network:

$$h_1 = f_1[x, \phi_1]$$

$$h_2 = f_2[h_1, \phi_2]$$

$$h_3 = f_3[h_2, \phi_3]$$

$$y = f_4[h_3, \phi_4]$$



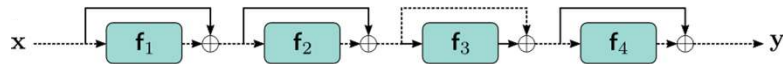
Residual network (2016):

$$h_1 = x + f_1[x, \phi_1]$$

$$h_2 = h_1 + f_2[h_1, \phi_2]$$

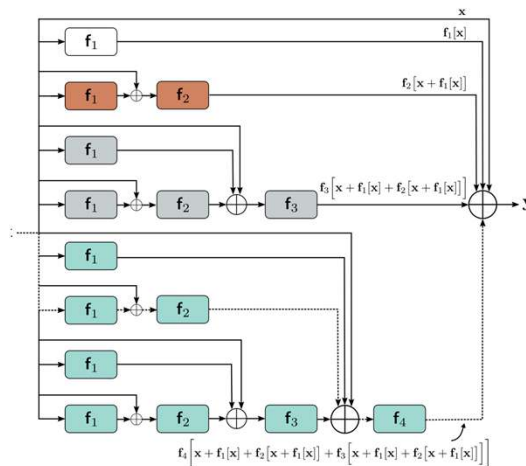
$$h_3 = h_2 + f_3[h_2, \phi_3]$$

$$y = h_3 + f_4[h_3, \phi_4]$$

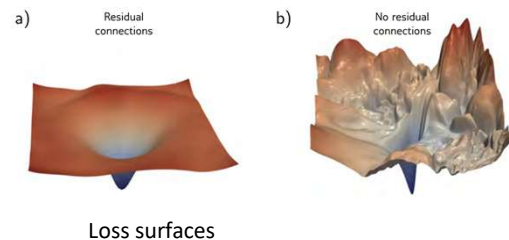


12

Resnet Unpacking

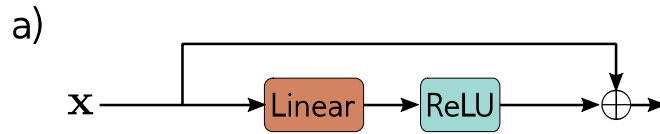


- Output is the sum of the input plus four smaller networks (white, orange, grey, and cyan)
- That is, output is an ensemble of networks
- Smaller networks do not lead to shattered gradients

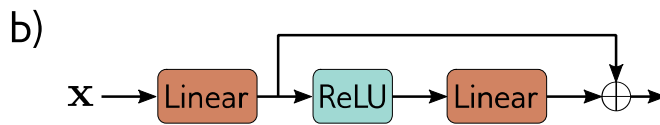


13

Order of operations



Each residual block can only increase the output since the output of ReLU is positive

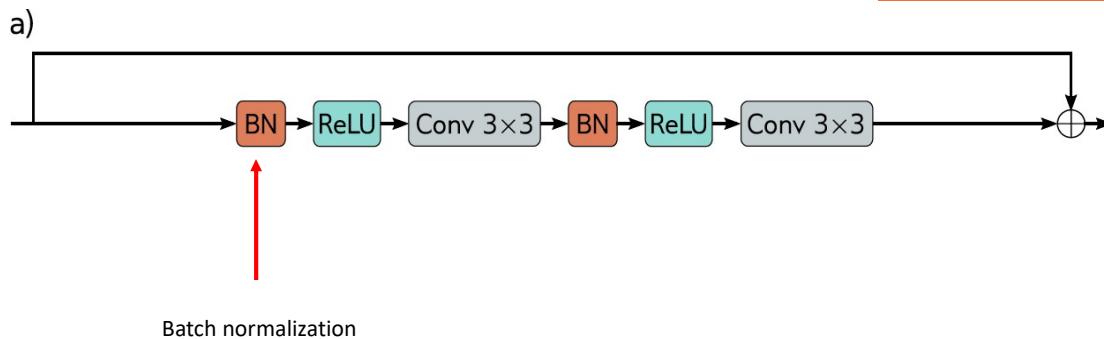


Interchanging the ReLU ensures that we can add both positive and negative quantities

Starting the network with a ReLU is problematic if the input is negative

14

Resnet Block



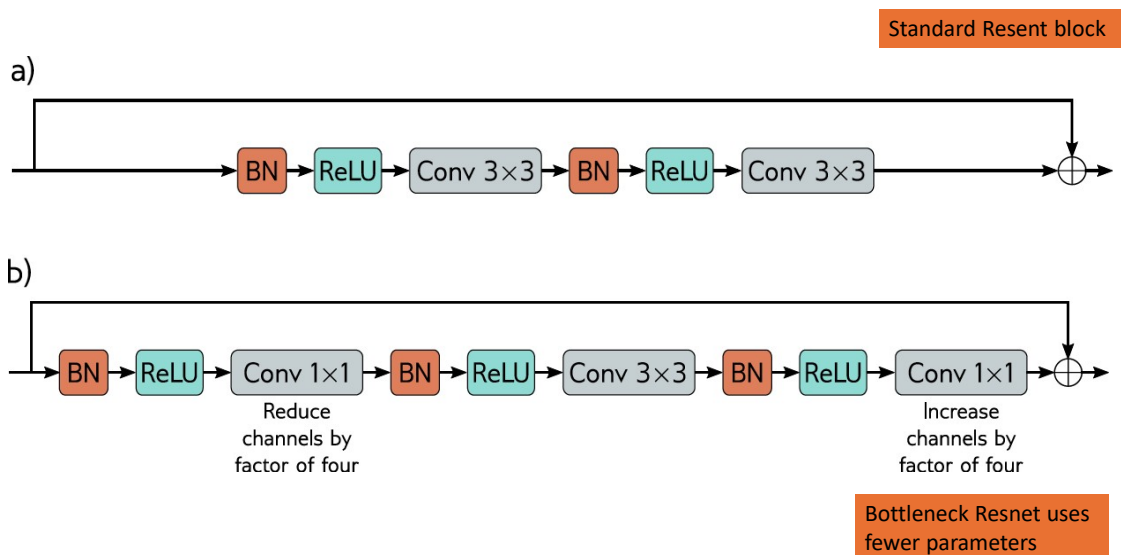
15

Batch Normalization

- Each activation in the batch is standardized with a z-score
 - A small value of epsilon is added if the standard deviation is zero
- The normalized variable is then scaled by gamma and shifted by delta
 - Gamma and delta are learned
- Batch normalization parameters are computed at each spatial location in a CNN
- At inference we have only 1 input, so the batch statistics are calculated over the entire input and frozen
- Benefits: stable forward propagation, higher learning rate, regularization

16

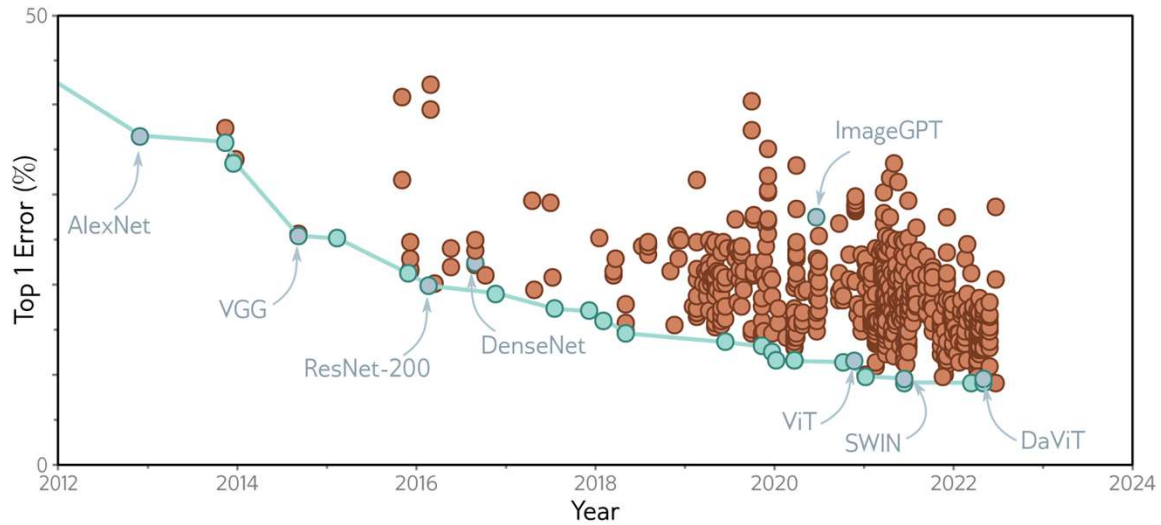
Resnet Block



17

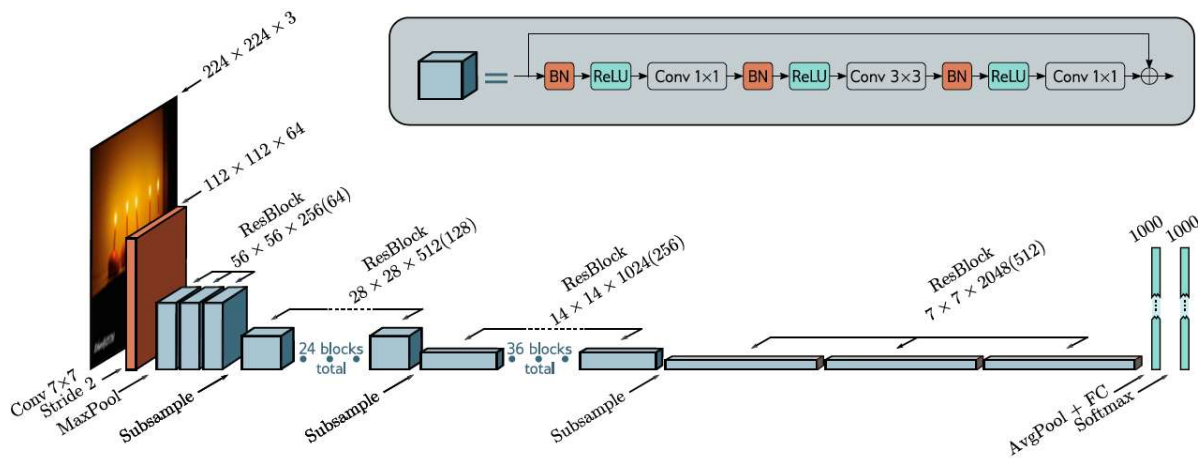
Resnet 200 (2016)

- 4.8% Top-5 error rate (resnet)
- ~5% Top-5 error rate (human)
- 20.1% Top-1 error rate



18

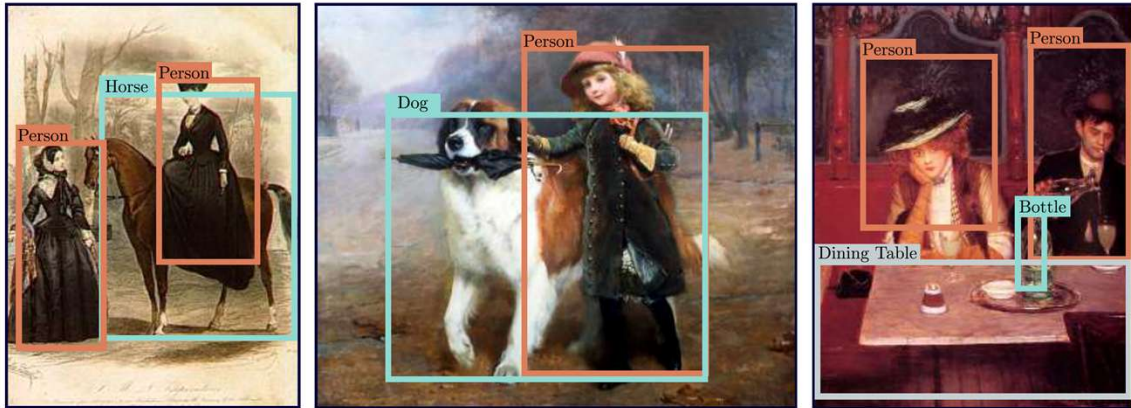
Resnet 200 (2016)



19

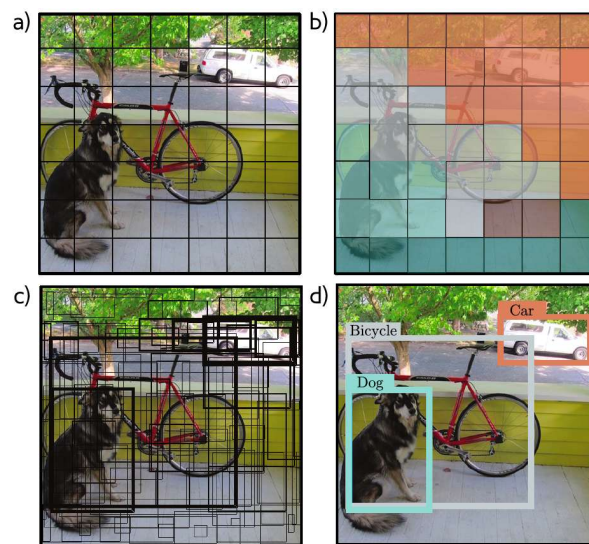
Object Detection

CNN arguably lifted the pessimism of Artificial Neural Network



20

Object detection (YOLO)



21

You Only Look Once (YOLO)

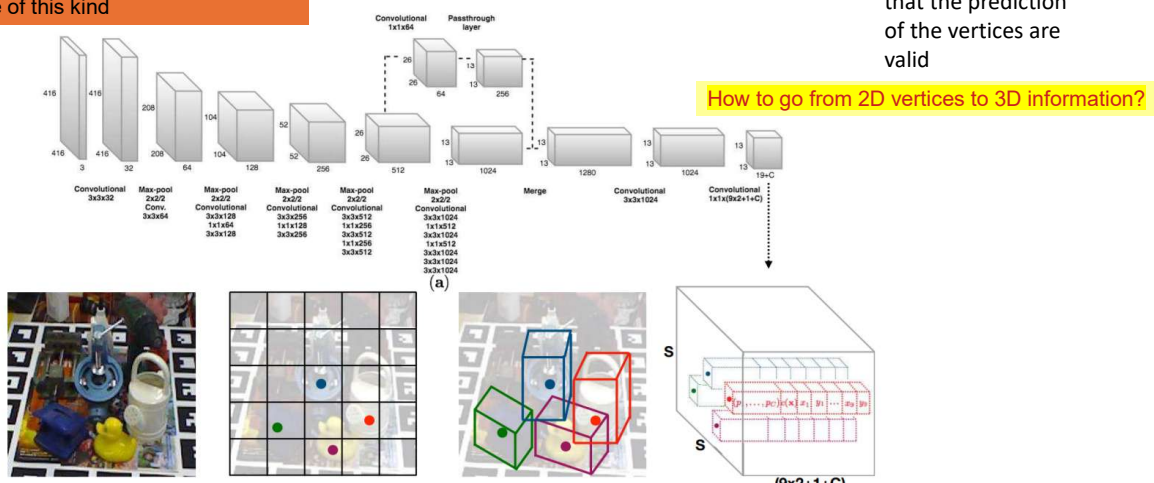
- Network similar to VGG
- 24 CNN layers, Input size: 448x448 input
- Final 7x7 layer has 1024 channels
- A FCN is then used to predict
 - Two bounding boxes at each location
 - Each bounding box has five parameters
 - x,y, height, width, and confidence
 - Example: yolov1 has $7 * 7 * (2*5 + 20)$, 20 is the number of classes
- Inference: Heuristic at the end to threshold and decide final boxes



22

Realtime Single Shot 6D Pose

Cannot understand this without a course of this kind



- Coordinates of 9 vertices
- C probabilities for each of the C classes
- 1 confidence value that the prediction of the vertices are valid

23